

Avaliação Final — IA Generativa (70% da nota final)

Visão Geral

Objetivo: Apresentar uma aplicação funcional com IA generativa integrada, demonstrando domínio sobre as decisões de engenharia de LLM: prompts, parâmetros, ferramentas (tools), escolha de framework e raciocínio por trás de cada decisão.

Data: 26/02/2026 (Aula 8) **Formato:** Individual — apresentação oral tipo "elevator pitch" **Duração por estudante:** 3 minutos de apresentação + 2 minutos de perguntas (5–7 min no total) **Tempo total estimado:** ~100–140 minutos

Descrição da Atividade

O que fazer

1. **Integrar IA generativa** em uma aplicação funcional. A sugestão é construir sobre o projeto da avaliação intermediária, mas **não é obrigatório** — pode ser um projeto novo.
2. **Usar qualquer ferramenta de codificação** (Claude Code, Codex, GitHub Copilot, Cursor) para auxiliar no desenvolvimento, inclusive na parte de integração com LLM.
3. **Tomar e documentar decisões de engenharia de LLM**, incluindo:
 - Escolha de framework (chamadas diretas à API vs. SDK vs. LangChain/LangGraph vs. outro)
 - Conteúdo e estrutura do system prompt
 - Parâmetros do modelo (temperatura, top-p, modelo escolhido, etc.)
 - Ferramentas (tools) disponibilizadas ao LLM e por quê
 - Estratégia de prompting (few-shot, chain-of-thought, XML tags, etc.)
 - Se aplicável: RAG, agentes, multi-agente, structured outputs
4. **Criar um repositório GitHub** com estrutura organizada (ver sugestão abaixo).
5. **Escrever um README detalhado** focado nas decisões de engenharia de LLM.
6. **Apresentar em 3 minutos** as escolhas feitas e estar preparado para responder perguntas.

Modelos de LLM — Pago vs. Local

Você tem total liberdade para escolher o modelo de LLM. Existem duas abordagens:

Abordagem	Exemplos	Quando faz sentido
APIs pagas	OpenAI (GPT-5), Anthropic (Claude), Google (Gemini)	Quando você precisa de máxima qualidade, tool calling robusto, ou context windows grandes
Modelos locais	Ollama ou vLLM com gpt-oss:20b, qwen3, nemotron-3-nano:30b	Quando você não pode ou não quer pagar, precisa de privacidade, ou quer experimentar modelos abertos

Independente da escolha, você deve documentar e justificar no README:

- Por que escolheu esse modelo/provedor?

- Quais são as limitações do modelo escolhido?
- Se usou modelo local: o que você espera que mudaria se alguém plugasse um modelo pago de maior capacidade?
- Se usou modelo pago: seria viável rodar com modelo local? O que se perderia?

Nenhum estudante será penalizado por usar modelo local em vez de pago. O que é avaliado é a **justificativa da escolha** e a **consciência das trade-offs** (custo, qualidade, latência, privacidade, capacidade de tool calling).

Estrutura sugerida do repositório

```
meu-projeto/  
├── README.md           # Documentação completa (ver seção abaixo)  
├── prompts/  
│   ├── system_prompt.txt # System prompt principal  
│   └── ...              # Outros prompts usados (few-shot examples, templates,  
etc.)  
├── tools/  
│   └── ...              # Definições de ferramentas disponibilizadas ao LLM  
├── agents/  
│   └── ...              # Lógica de agentes (se aplicável)  
├── src/ ou app/  
│   └── ...              # Código da aplicação  
└── ...
```

Essa estrutura não é obrigatória, mas facilita a avaliação. O importante é que prompts, tools e agentes estejam facilmente localizáveis no repositório.

A Apresentação (Elevator Pitch)

Você tem **3 minutos**. Foque no que importa:

1. **O que o sistema faz** — em uma frase (30s)
2. **Decisões de LLM** — a parte principal da apresentação (2min):
 - Que modelo/API usa e por quê?
 - Framework: chamadas diretas, SDK, LangChain? Por quê?
 - O que está no system prompt e por quê?
 - Que parâmetros configurou e por quê? (temperatura, etc.)
 - Que ferramentas (tools) o LLM tem acesso? Por quê essas?
 - Que técnica de prompting usou?
3. **O que funcionou e o que não funcionou** (30s)

Após os 3 minutos, o professor fará **2 minutos de perguntas** sobre suas escolhas. Exemplos de perguntas:

- "Por que você usou temperatura 0.7 e não 0?"
- "O que acontece se o usuário enviar um input malicioso?"
- "Por que LangChain e não chamada direta à API?"

- "O que esse trecho do system prompt faz?"
- "Se eu mudar esse parâmetro, o que muda no comportamento?"

Aviso: É permitido usar ferramentas de codificação com IA para desenvolver todo o projeto. Porém, você **precisa entender e justificar** cada decisão de engenharia de LLM. Se o agente de codificação montou tudo e você não souber explicar por que o system prompt diz o que diz ou por que uma ferramenta foi definida daquele jeito, isso ficará evidente nas perguntas — mesmo em 2 minutos.

Critérios de Avaliação

A nota da avaliação final é composta por **70 pontos** (equivalentes aos 70% da nota final).

1. System Prompt e Estratégia de Prompting (18 pontos)

Pontos	Critério
18	System prompt bem estruturado, com instruções claras, persona definida, restrições de comportamento, formato de saída especificado. Uso efetivo de técnicas (few-shot, CoT, XML tags, etc.). Demonstra iteração e refinamento
14	System prompt funcional e bem pensado, com a maioria dos elementos acima. Boa estratégia de prompting
10	System prompt razoável mas genérico ou incompleto. Alguma técnica de prompting aplicada
6	System prompt básico, sem estrutura clara. Pouca evidência de engenharia de prompts
0	Sem system prompt ou prompt trivial ("Você é um assistente útil")

2. Ferramentas (Tools) e Integração (14 pontos)

Pontos	Critério
14	Ferramentas bem definidas, com descrições claras para o modelo, parâmetros tipados, tratamento de erros. Justificativa clara de por que cada ferramenta existe. Integração coerente com o fluxo da aplicação
10	Ferramentas funcionais e relevantes, boa integração, descrições adequadas
7	Ferramentas básicas, funcionam mas sem refinamento nas descrições ou na integração
3	Ferramentas mínimas ou mal definidas
0	Sem ferramentas ou ferramentas que não funcionam

3. Escolha e Configuração de Parâmetros (10 pontos)

Pontos	Critério
10	Escolha deliberada de modelo, temperatura, top-p e outros parâmetros, com justificativa clara. Evidência de experimentação (testou diferentes valores). Parâmetros adequados ao caso de uso

Pontos	Critério
7	Parâmetros configurados de forma razoável com alguma justificativa
4	Parâmetros configurados mas sem justificativa clara ou com valores padrão sem reflexão
0	Parâmetros padrão sem nenhuma consideração

4. Arquitetura e Escolha de Framework (10 pontos)

Pontos	Critério
10	Escolha justificada entre API direta / SDK / framework. Arquitetura coerente (se usa agentes, se usa RAG, se é multi-agente). Trade-offs considerados (complexidade vs. funcionalidade, custo vs. qualidade)
7	Escolha razoável com alguma justificativa. Arquitetura funcional
4	Framework usado sem justificativa clara ou arquitetura confusa
0	Sem reflexão sobre escolha de abordagem

5. README e Documentação (10 pontos)

O README deve focar nas decisões de engenharia de LLM:

Pontos	Critério
2	Descrição do problema e da solução — O que o sistema faz? Como a IA é usada?
2	Arquitetura de LLM — Diagrama ou descrição do fluxo: input do usuário → prompt → modelo → tools → resposta
2	Decisões e justificativas — Por que esse modelo? Por que esses parâmetros? Por que essas ferramentas?
2	O que funcionou — Quais decisões de prompting/ferramentas deram bons resultados?
2	O que não funcionou — Onde o LLM falhou? Que ajustes foram tentados? Limitações encontradas?

6. Apresentação Oral e Respostas (8 pontos)

Pontos	Critério
8	Apresentação clara e objetiva dentro do tempo. Respostas às perguntas demonstram compreensão profunda das escolhas feitas. Sabe explicar o "porquê" de cada decisão
6	Boa apresentação. Responde a maioria das perguntas com segurança
4	Apresentação razoável. Dificuldade em justificar algumas decisões
2	Apresentação vaga. Não consegue explicar decisões-chave — sugere que não entende o próprio código

Pontos	Critério
0	Não apresentou

Resumo da Pontuação

Critério	Pontos
System prompt e estratégia de prompting	18
Ferramentas (tools) e integração	14
Escolha e configuração de parâmetros	10
Arquitetura e escolha de framework	10
README e documentação	10
Apresentação oral e respostas	8
Total	70

Entregáveis

1. **Link do repositório GitHub** (público ou com acesso concedido ao professor)
2. **README.md** com documentação focada em decisões de engenharia de LLM
3. **Prompts, tools e agentes** claramente organizados no repositório
4. **Apresentação oral** de 3 minutos na Aula 8

Nota: Diferente da avaliação intermediária, o endpoint não precisa estar publicamente acessível no dia — mas se estiver, pode ser usado como apoio durante a apresentação.

Dicas

- **Construa sobre a avaliação intermediária:** Se a sua UI já está pronta, o foco agora é toda a engenharia de LLM. Isso poupa tempo.
- **Separe seus prompts em arquivos:** Manter o system prompt em `prompts/system_prompt.txt` facilita iterar e versionar.
- **Experimente parâmetros:** Teste diferentes temperaturas, modelos, estratégias. Documente o que mudou no comportamento.
- **Prepare-se para as perguntas:** Se você não sabe explicar por que seu system prompt tem determinada instrução, revise. O professor vai perguntar.
- **3 minutos é pouco:** Ensaie. Não gaste tempo explicando a UI — foque nas decisões de LLM.
- **Ferramentas de codificação são aliadas, não substitutas:** Use Claude Code, Copilot, etc. para montar tudo — mas entenda o que foi montado.

FAQ

P: Preciso construir sobre o projeto da avaliação intermediária? R: Não. É recomendado (porque a UI já estará pronta), mas você pode fazer um projeto novo.

P: Posso usar qualquer modelo de LLM? R: Sim. Veja a seção "Modelos de LLM" acima. Tanto APIs pagas quanto modelos locais são aceitos — o importante é justificar a escolha.

P: E se eu usar RAG ou agentes? R: Excelente — mas precisa justificar por que RAG ou agentes são necessários para o seu caso de uso, não apenas "porque vimos em aula".

P: Como o professor vai avaliar os prompts e ferramentas? R: O professor vai ler o README, examinar os arquivos em `prompts/`, `tools/` e `agents/`, e fazer perguntas na apresentação para verificar se você entende as escolhas.

P: Posso usar o agente de codificação para montar o system prompt e as ferramentas? R: Sim, mas você precisa entender e justificar o resultado. "O Claude Code gerou assim" não é uma justificativa válida.